Paradigmas de Programación II

Ingeniería en Informática
Ingeniería en Electrónica
Ingeniería en Telecomunicaciones

Parte I

Interacción Básica entre Objetos

Conceptos Fundamentales

- Abstracción
- Modularización
- Las clases definen tipos
- Diagrama de clases
- Diagrama de objetos
- Referencias a objetos
- Tipos primitivos
- Tipos objeto
- Creación de objetos
- Sobrecarga
- Llamada a métodos internos/externos
- Depurador

Abstracción y Modularización

- La Abstracción es la habilidad de ignorar los detalles de las partes para enfocar la atención en un nivel más elevado de un problema.

- La Modularización es el proceso de dividir un todo en partes bien definidas que interactúan en formas bien definidas. Estas partes pueden construirse y examinarse en forma separada.

Ejemplo: Proyecto Reloj Digital

Se desea construir un reloj digital.

El reloj muestra las horas y los minutos.

El rango del reloj es de 00:00 (medianoche) a 23:59 (un minuto antes de la medianoche).

Cada minuto, el reloj debe actualizar la hora, y la pantalla muestra la hora (incrementa minuto u hora).

La hora se incrementa cuando los minutos llegan a cero.

Ejemplo: Proyecto Reloj Digital

Formato reloj:

- **Horas**
- **Minutos**

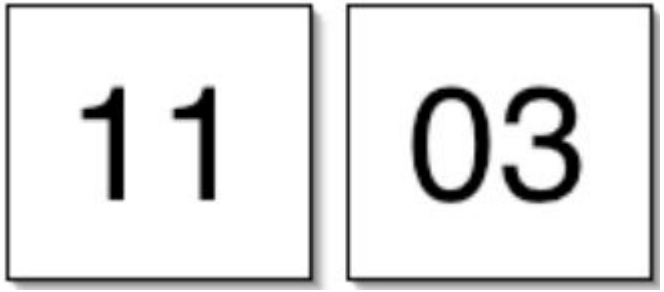
Nos concentramos en el diseño y construcción del display del Reloj.

Ejemplo: Proyecto Reloj Digital

Modularización del visor del reloj.

¿Consideramos un único módulo?

¿Podemos trabajar con varios módulos interactuando entre si?



Implementación del visor de números

numeroDisplay es la clase que representa un visor de números general, que se utiliza par construir los objetos representativos de horas y minutos.

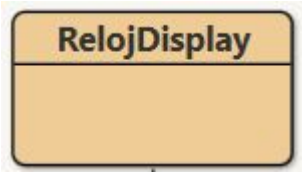
```
public class NumeroDisplay
{
   private int limite;
   private int valor;

   public NumeroDisplay(int limite)
   {
      this.limite = limite;
      this.valor = 0;
   }
....
}
```

Implementación visor del reloj

El display del reloj se compone de dos visores de números (displayNumero).

Uno para representar las horas.

El otro para representar los minutos.




```
public class RelojDisplay
{
   private NumeroDisplay horas;
   private NumeroDisplay minutos;
   private String displayString;    //Simular el
display

   public RelojDisplay()
   {
      horas = new NumeroDisplay(24);
      minutos = new NumeroDisplay(60);
      actualizarDisplay();
   }
```

Diagrama de Objetos

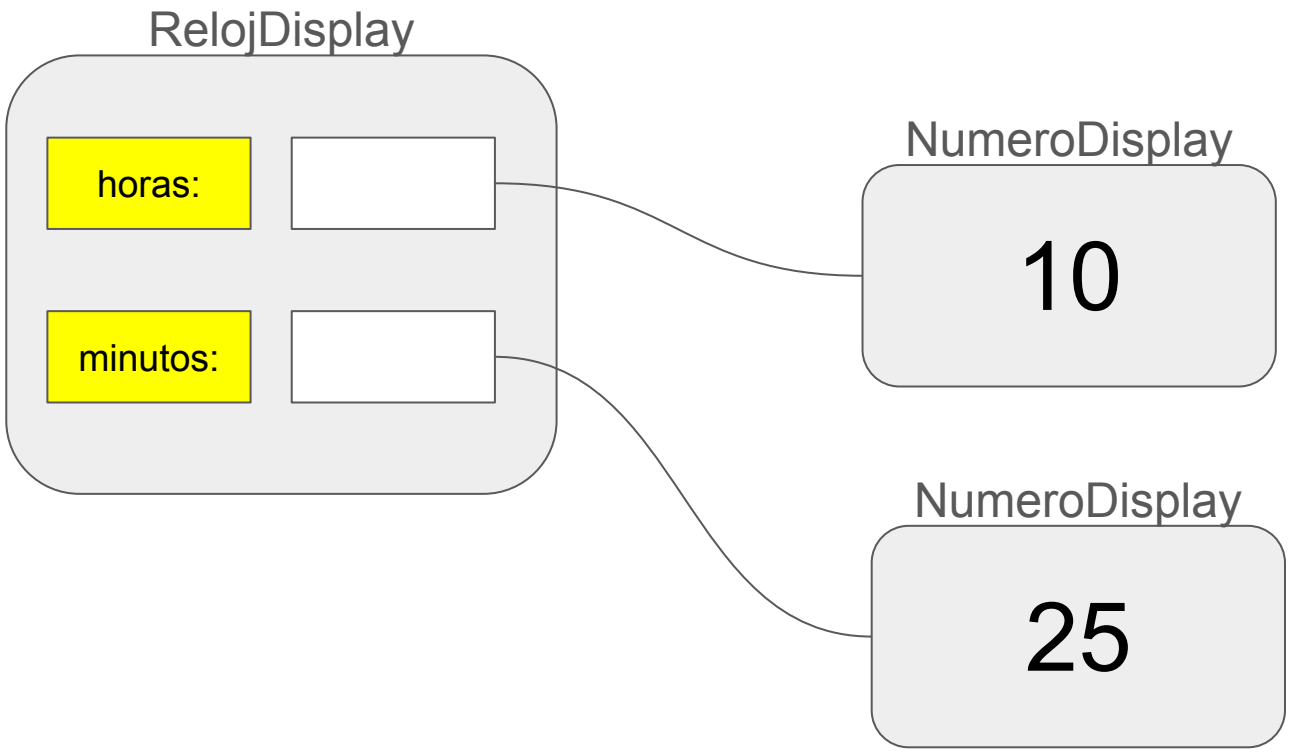
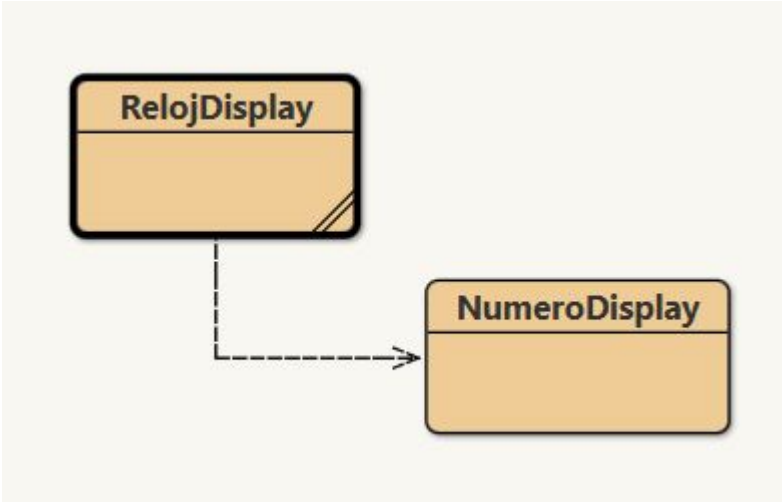


Diagrama de Clases

Consideramos diseñar el Display del reloj conformado por dos visores (numeroDisplay).

- Visor para minutos
- Visor para horas

El diagrama de clases muestra la forma en que se relacionan las clases.



Código fuente

Clase NumeroDisplay

- **public NumeroDisplay(int limite):** Constructor. Parámetro límite define el máximo valor a representar
- **public int getValor():** Método getter. Permite obtener el valor del visor.
- **public String getValorDisplay():** Método getter. Permite obtener el valor del visor con formato de dos dígitos.
- **public void setValor(int valor):** Método setter. Setea el valor del visor a un valor determinado por parámetro.
- **public void incremento():** Método que incrementa el valor del visor. Cuando se alcanza el límite se vuelve el valor a 0.

Código fuente

Clase RelojDisplay

- **public RelojDisplay():** Constructor default del Display del reloj.
- **public RelojDisplay(int horas, int minutos):** Constructor con parámteros del Display del reloj.
- **public void incrementoTiempo():** Método que incrementa la hora del reloj.
- **public void setHora(int horas, int minutos):** Método setter para establecer la hora a un valor determinado.
- **public String getHora():** Método getter para retornar la hora.
- **private void actualizarDisplay():** Método que actualiza el display del reloj en base a los valores que figuran en cada unos de los visores.

Objetos que crean objetos

```
public class RelojDisplay
{
    private NumeroDisplay horas;
    private NumeroDisplay minutos;
    private String displayString;    // Solo para
simular el display actual

    /**
     * Constructor RelojDisplay.
     * Crea un nuevo reloj seteado a 00:00.
     */
    public RelojDisplay()
    {
        horas = new NumeroDisplay(24);
        minutos = new NumeroDisplay(60);
        actualizarDisplay();
    }
```

En la clase **RelojDisplay**, se establecen atributos **horas** y **minutos** del tipo de la clase **NumeroDisplay**.

Esto implica que el objeto **RelojDisplay** crea objetos del tipo **NumeroDisplay**.

Llamadas a Métodos

```
public void incrementoTiempo()
{
    minutos.incremento();
    if(minutos.getValor() == 0) {
        horas.incremento();
    }
    actualizarDisplay();
}
```

Método incrementoTiempo debe ser llamado cada un minuto.
Esto hace que el display de minutos avance una unidad.

Verificar que el método **incrementoTiempo** realiza una llamada a otro método **actualizarDisplay**.

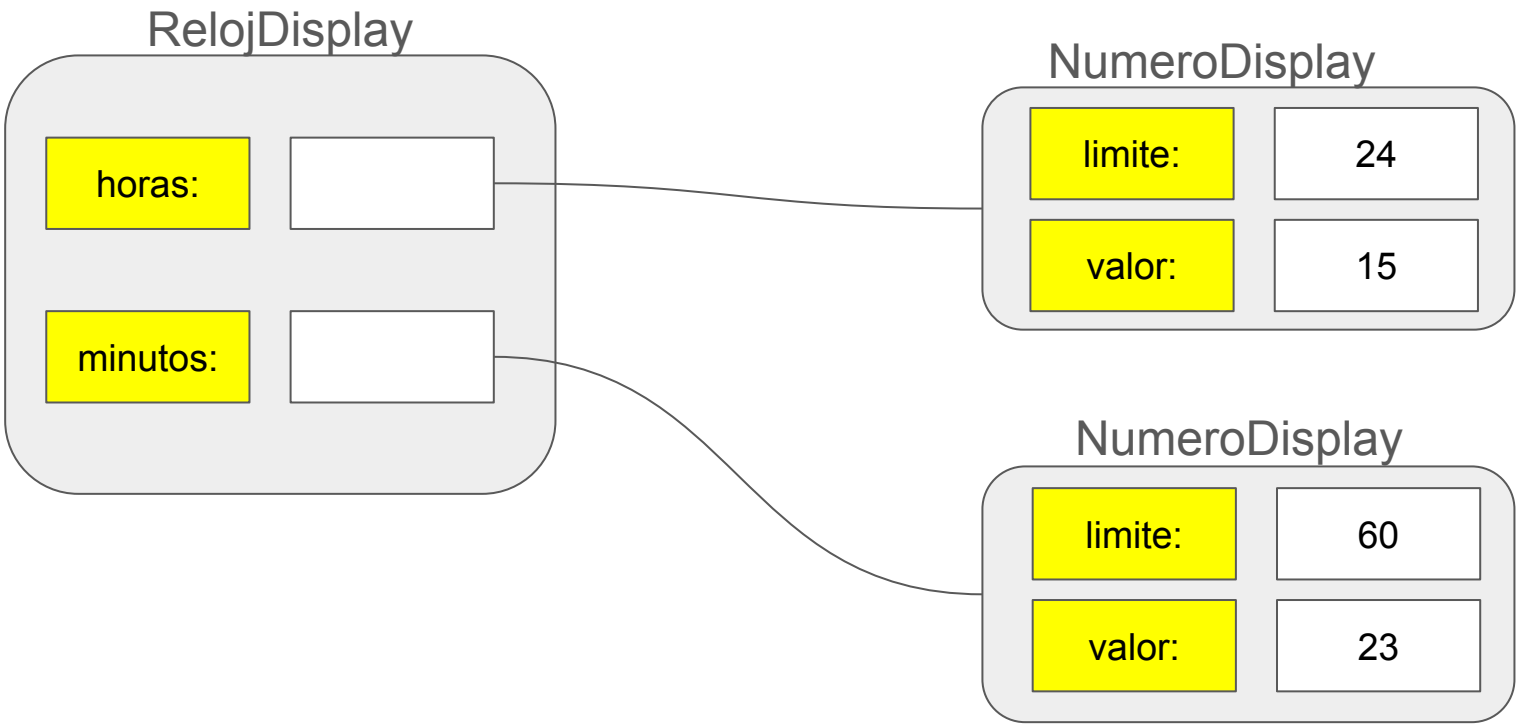
Método Interno

```
private void actualizarDisplay()
   {
      displayString = horas.getValorDisplay() + ":" +
                     minutos.getValorDisplay();
   }
```

Método de la clase que se encarga de actualizar que representa la cadena de visualización.

Este método es invocado **internamente** por los otros métodos de la clase. En este caso por el método **incrementoTiempo**

Diagrama de objetos del RelojDisplay



Objetos que crean objetos

```
/**
    * Constructor para objetos de la clase NumeroDisplay.
    * Establece el límite en el que se reinicia la visualización.
    */
   public NumeroDisplay(int limite)
   {
      this.limite = limite;
      this.valor = 0;
   }
```

Parámetro Formal

```
/**
    * Constructor RelojDisplay.
    * Crea un nuevo reloj seteado a horas y minutos especificada como parámetros.
    */
   public RelojDisplay(int horas, int minutos)
   {
      this.horas = new NumeroDisplay(24);
      this.minutos = new NumeroDisplay(60);
      setHora(horas, minutos);
   }
```

Parámetro Real

Llamadas a métodos

- Llamadas a métodos internos:

  actualizarVisor(); ...

  private void actualizarVisor() // Solo accesible para otros métodos de la clase

- Llamadas a métodos externos:

  minutos.incrementar();

- Sintaxis para llamada a métodos de otros objetos:

  objeto.nombreMetodo (lista-parámetros)

Tipos de datos Java

- El sistema de tipos de Java está basado en dos clases distintas de tipos: **tipos primitivos y tipos de objeto**.
- Los tipos primitivos se almacenan directamente en variables y están dotados de una semántica de valor (los valores se copian al ser asignados a otra variable). Los tipos primitivos no están asociados con clases y no tienen métodos.
- En cambio, un tipo de objeto se manipula almacenando una referencia al objeto (no al propio objeto). Cuando se asigna a una variable, solo se copia la referencia, no el objeto.

Tipos de datos Java

Dos categorías de tipos en Java

En Java existen dos grupos principales:

1. **Tipos primitivos**

2. **Tipos de objeto (o de referencia)**

Tipo	Qué guarda la variable	Ejemplo
Primitivo	El **valor directamente**	`int x = 10;`
Objeto	Una **referencia (dirección)** al objeto	`String nombre = "Juan";`

Tipos de datos Java

Tipos primitivos

Los **tipos primitivos** almacenan el **valor directamente dentro de la variable**.

```java
int a = 5;
int b = a;

b = 10;

System.out.println(a); // 5
System.out.println(b); // 10
```

Tipos de datos Java

Características de los tipos primitivos

- Guardan **el valor directamente**
- Se copian **por valor**
- No tienen métodos
- No son objetos

Ejemplos de tipos primitivos

Tipo	Ejemplo
`int`	`int edad = 20;`
`double`	`double precio = 15.5;`
`char`	`char letra = 'A';`
`boolean`	`boolean activo = true;`

Tipos de datos Java

Tipos de objeto (tipos de referencia)

- Los **objetos no se guardan directamente en la variable**.
- La variable guarda **una referencia (una dirección en memoria)** al objeto.

String es un caso particular

```java
String a = "Hola";
String b = a;

b = "Adios";

System.out.println(a); // Hola
System.out.println(b); // Adios
```

Tipos de datos Java

4. Ejemplo claro con objetos

Supongamos una clase:

```java
class Persona {
    String nombre;
}
```

```java
Persona p1 = new Persona();
p1.nombre = "Juan";

Persona p2 = p1;

p2.nombre = "Pedro";

System.out.println(p1.nombre);
```

Devuelve “Pedro”

Cuando hicimos:

Persona p2 = p1;

No se copió el objeto.
Se copió **la referencia**.

Representación en memoria:

p1 ----\
----> objeto Persona { nombre = "Juan" }
p2 ----/

Tipos de datos Java

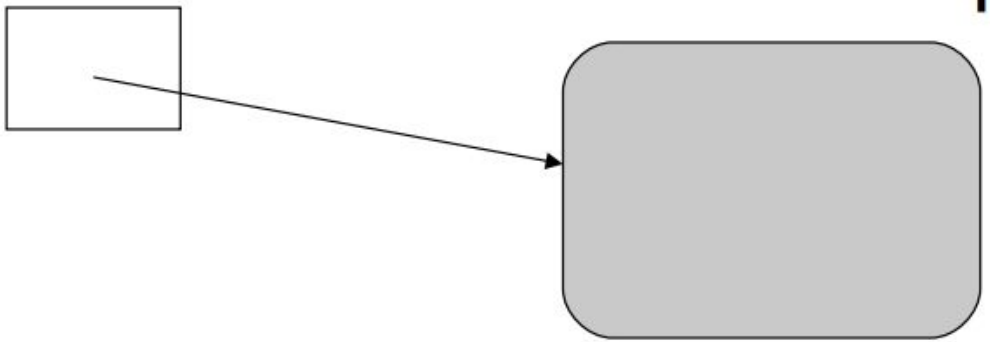
Resumen Conceptual

Característica	Primitivos	Objetos
Qué guarda la variable	valor	referencia
Copia en asignación	valor	referencia
Métodos	no	sí
Ejemplos	int, double, boolean	String, ArrayList, clases

Tipos primitivos y tipos objeto

```
AlgunObjeto obj;
```

tipo de objeto

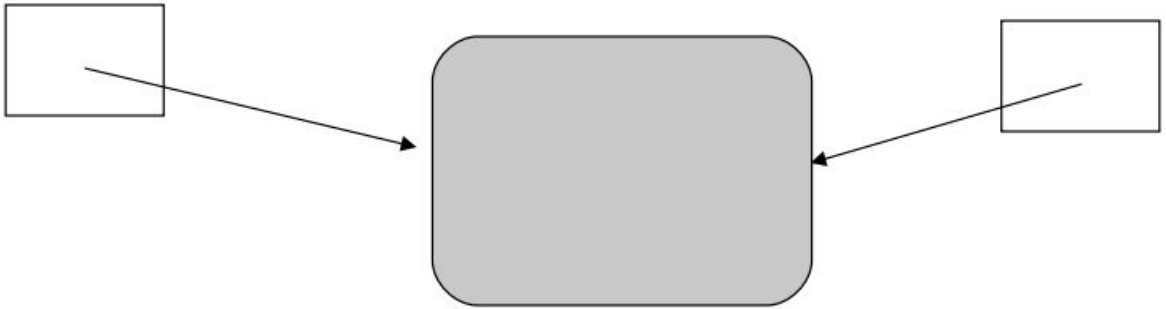


```
int i;
```

tipo primitivo

Tipos primitivos y tipos objeto

```
TipoDeObjeto a;        TipoDeObjeto b;
```



```
b = a;
```

```
int a;                 int b;
```